AWS VPC Peering

Connect two VPCs. Prove private connectivity.

Field	Detail
Name	Jinu G Mathew
Batch	AWS CLOUD INTERNSHIP
Project title	EWB Project 01: VPC Peering (AWS Architect Associate)
Region	Asia Pacific (Mumbai) / ap-south-1
Completion date	21-10-2026
Tags used	Project = EWB-VPC-Peering, Student = Jinu G Mathew

1. Architecture and resource inventory

Two VPCs with non-overlapping ranges are connected by a peering connection. VPC A holds a public subnet (bastion) and a private subnet (App-A). VPC B holds one private subnet (App-B) and has no internet gateway. Administration uses SSH from my laptop through the bastion (ProxyJump); project traffic between App-A and App-B uses private IPs over the peering connection. The two VPC ranges (10.10.0.0/16 and 10.20.0.0/16) do not overlap, which AWS requires before a peering connection can be created.

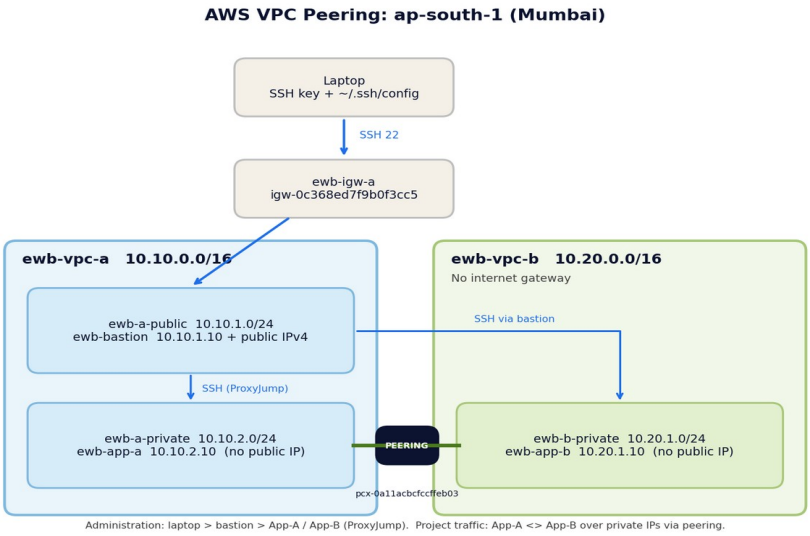


Figure 1: Architecture: laptop, internet gateway, VPC A, VPC B and the peering link.

Resource inventory

Resource	ID	CIDR / detail
ewb-vpc-a	vpc-0cb895b79d1f4d9e9	10.10.0.0/16
ewb-a-public	subnet-0cf719c9cde8a0d1a	10.10.1.0/24, ap-south-1a
ewb-a-private	subnet-07b349789b6514154	10.10.2.0/24, ap-south-1a
ewb-igw-a	igw-0c368ed7f9b0f3cc5	Attached to ewb-vpc-a
ewb-rt-a-public	rtb-0384fbcd0a677943b	Associated with ewb-a-public
ewb-rt-a-private	rtb-01663791c7be2fe84	Associated with ewb-a-private
ewb-vpc-b	vpc-020f50e2d7f986ae8	10.20.0.0/16
ewb-b-private	subnet-05460b0d2112b9c04	10.20.1.0/24, ap-south-1a
ewb-rt-b-private	rtb-0574c4f332ac00da0	Associated with ewb-b-private
ewb-sg-bastion	sg-04515ecee1fe68bc6	VPC A, SSH 22 from my IP /32
ewb-sg-app-a	sg-04118062cd1d94c24	VPC A, SSH from bastion, ICMP Echo Request from App-B
ewb-sg-app-b	sg-05bd6640f3ba9a70d	VPC B, SSH from bastion, ICMP Echo Request from App-A
ewb-lab (key pair)	key-0a49002c97fab867e	RSA, .pem
ewb-bastion	i-08206747569707b4a	10.10.1.10, public IPv4 3.110.193.249
ewb-app-a	i-00a28dddb86cbc8f2	10.10.2.10, no public IP
ewb-app-b	i-069b23803f34af327	10.20.1.10, no public IP

Resource	ID	CIDR / detail
ewb-a-b-peer	pcx-0a11acbcfccffeb03	ewb-vpc-a (requester) to ewb-vpc-b (accepter)

2. VPC A and its subnets

2.1 Create ewb-vpc-a

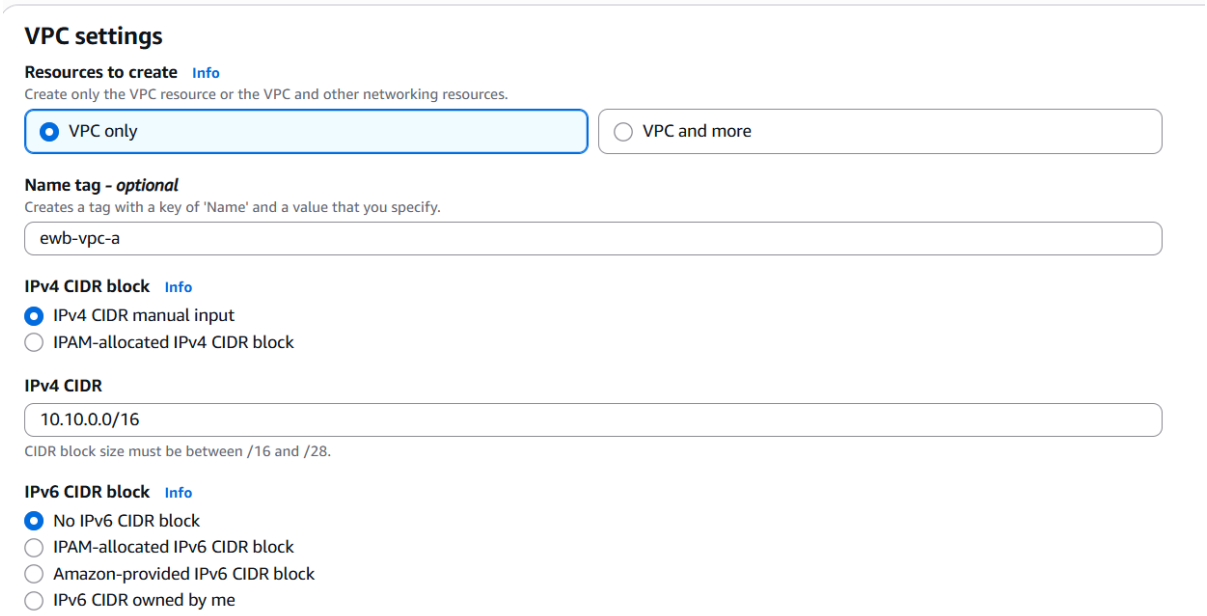


Figure 2: VPC creation settings for ewb-vpc-a: VPC only, IPv4 manual input 10.10.0.0/16, no IPv6, tenancy default.

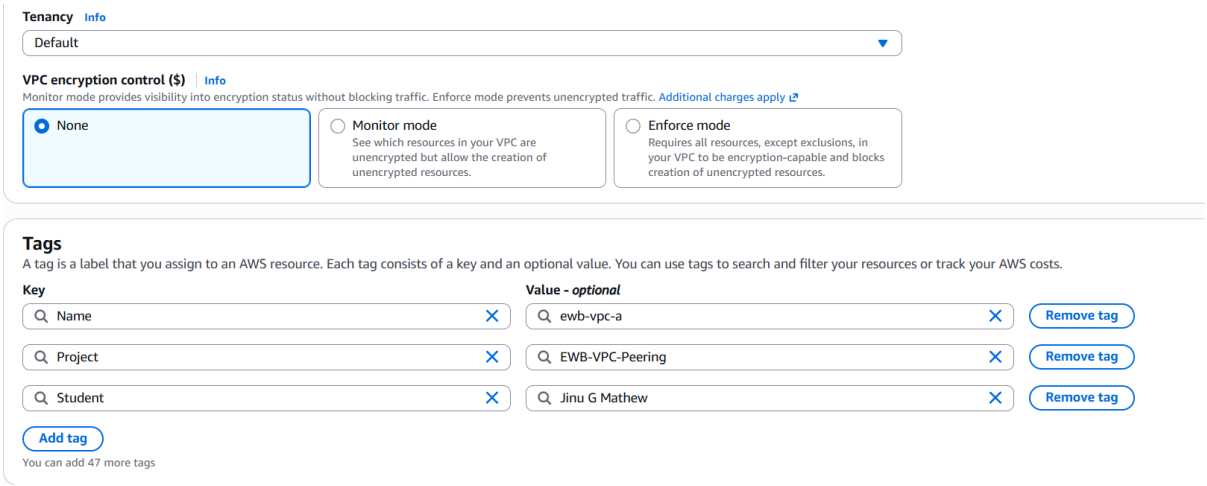


Figure 3: Tenancy default, encryption control set to None, and the Name, Project and Student tags.

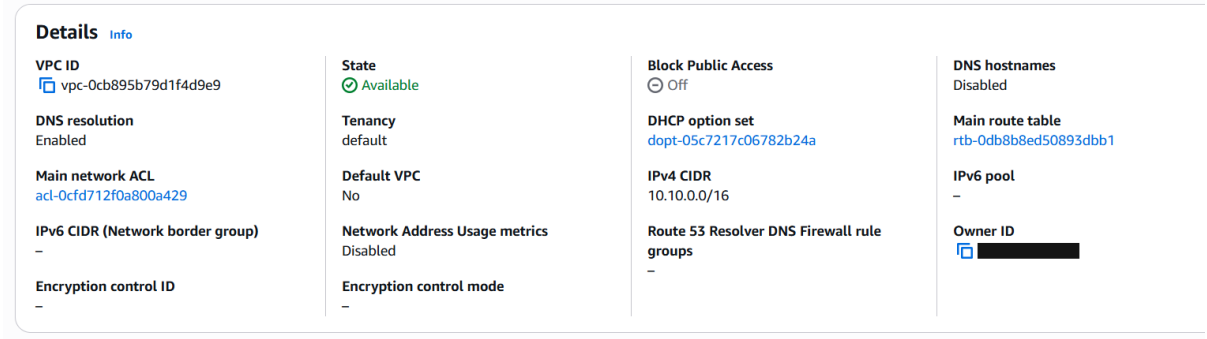


Figure 4: ewb-vpc-a (vpc-0cb895b79d1f4d9e9) created. State Available, IPv4 CIDR 10.10.0.0/16, not the default VPC.

2.2 Create the subnets

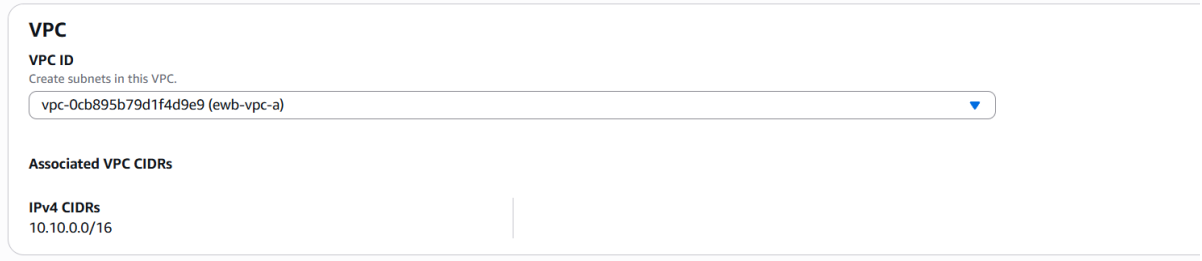


Figure 5: Subnets are created inside vpc-0cb895b79d1f4d9e9 (ewb-vpc-a), CIDR 10.10.0.0/16.

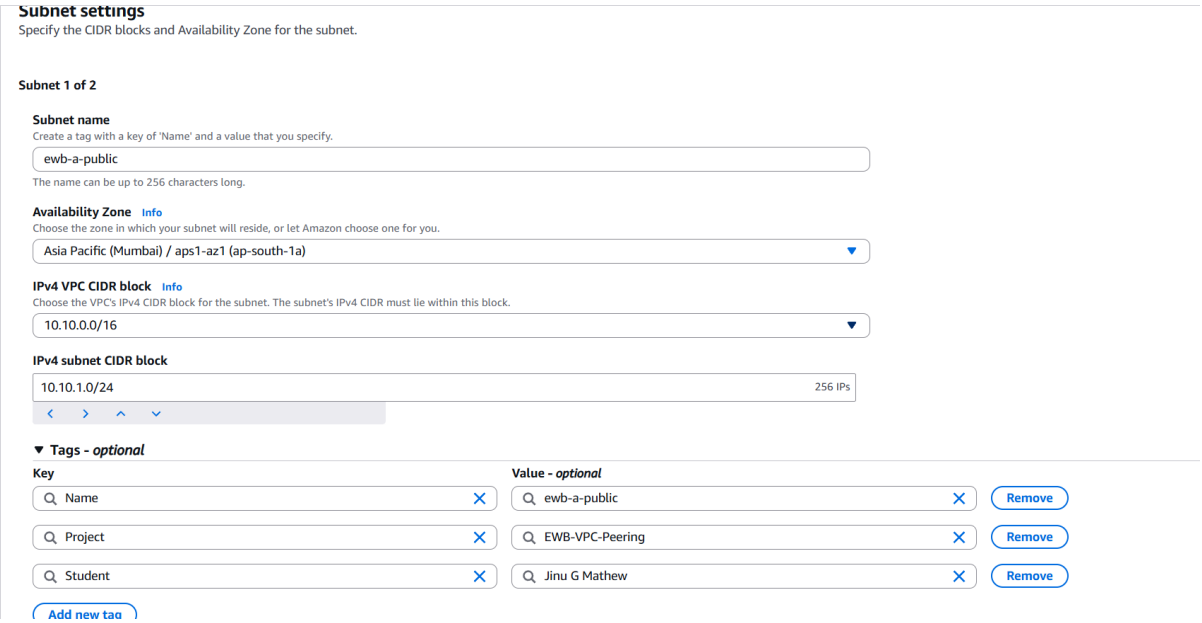


Figure 6: Subnet ewb-a-public: ap-south-1a, 10.10.1.0/24, with Name, Project and Student tags.

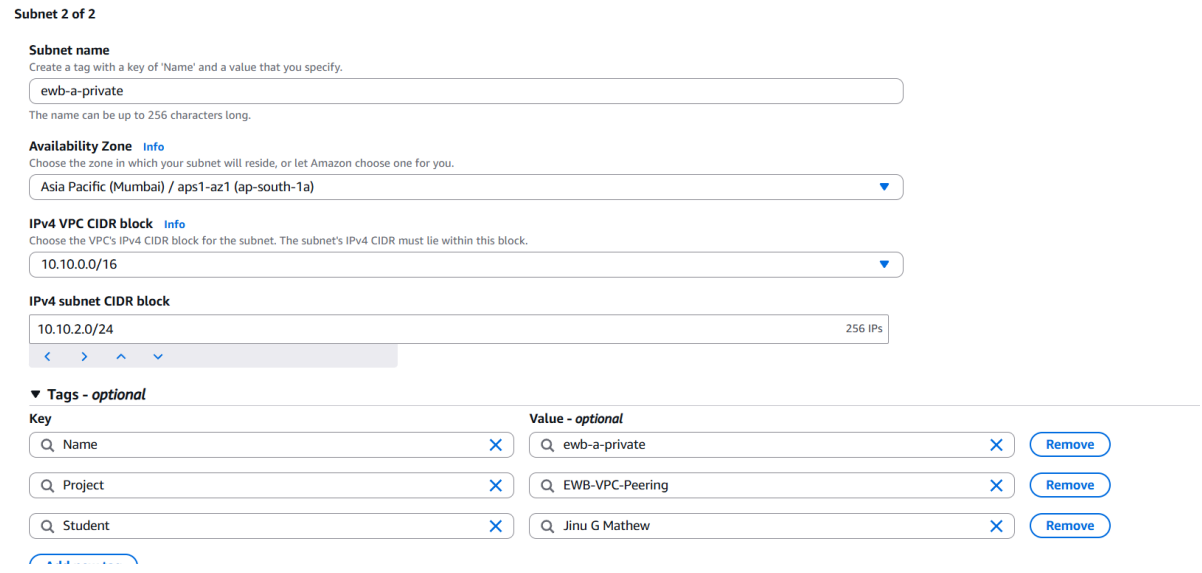


Figure 7: Subnet ewb-a-private: ap-south-1a, 10.10.2.0/24, with Name, Project and Student tags.

3. Internet gateway and route tables for VPC A (Checkpoint 01)

3.1 Internet gateway

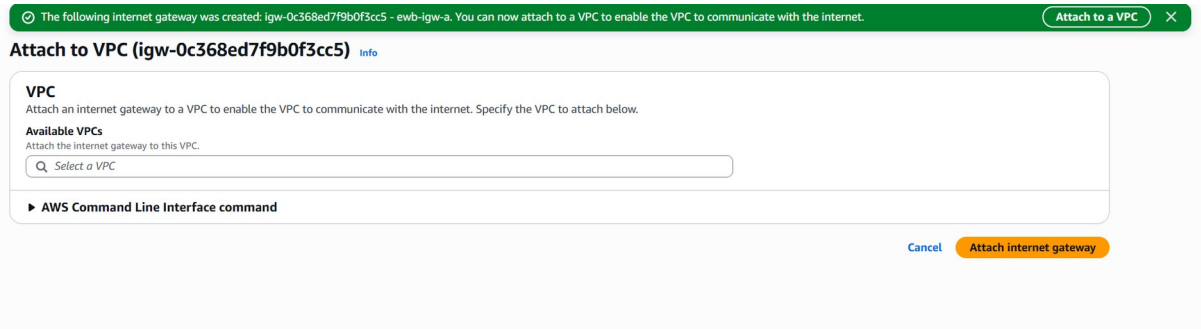


Figure 8: ewb-igw-a (igw-0c368ed7f9b0f3cc5) created and ready to attach to ewb-vpc-a.

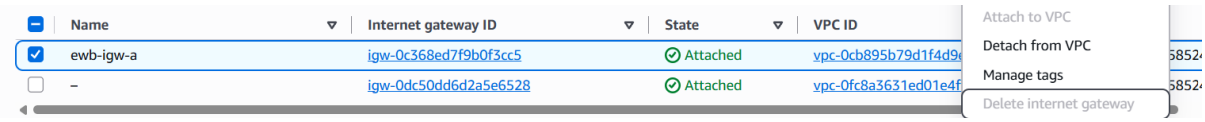


Figure 9: Internet gateways list: ewb-igw-a is Attached to the VPC A (vpc-0cb895b79d1f4d9e9). The second gateway belongs to the default VPC.

3.2 Route tables

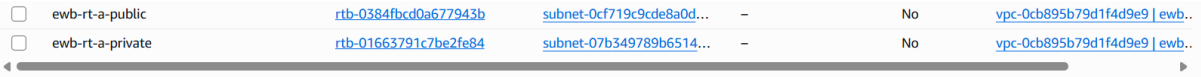


Figure 10: ewb-rt-a-public (rtb-0384fbcd0a677943b) and ewb-rt-a-private (rtb-01663791c7be2fe84) both belong to vpc-0cb895b79d1f4d9e9.

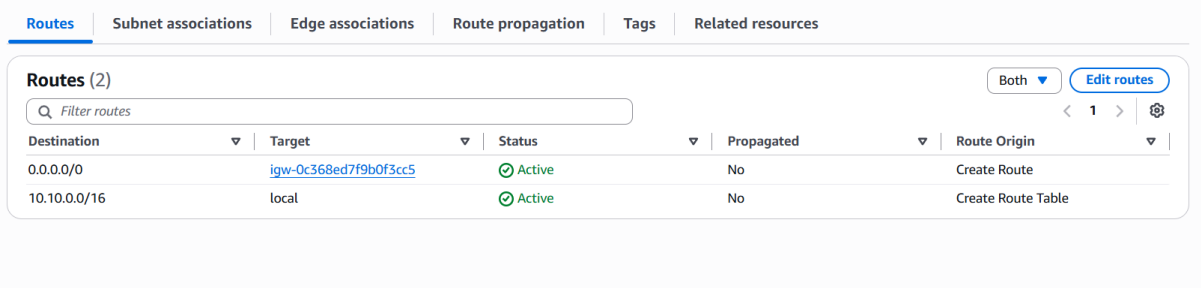


Figure 11: ewb-rt-a-public, Routes tab: 10.10.0.0/16 local and 0.0.0.0/0 to igw-0c368ed7f9b0f3cc5. This default route makes ewb-a-public a public subnet.

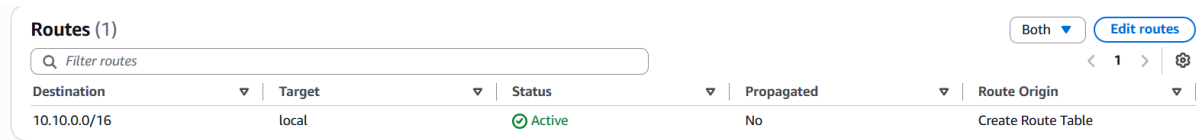


Figure 12: ewb-rt-a-private, Routes tab (before peering): only 10.10.0.0/16 local. No internet route, so ewb-a-private is private.

The explicit subnet associations are shown in the Details panel of each route table in Section 10: ewb-rt-a-public is associated with ewb-a-public, and ewb-rt-a-private with ewb-a-private.

4. VPC B

4.1 Create ewb-vpc-b

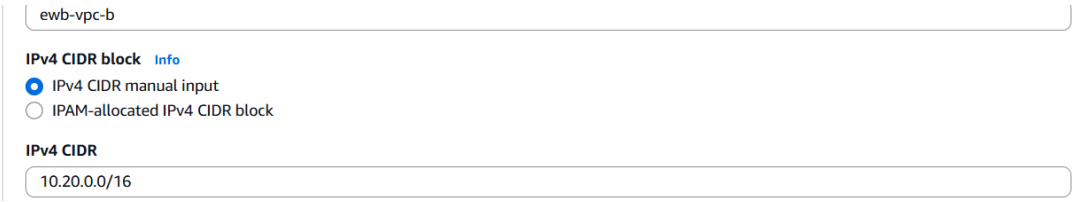


Figure 13: ewb-vpc-b uses IPv4 manual input with CIDR 10.20.0.0/16, which does not overlap with VPC A.

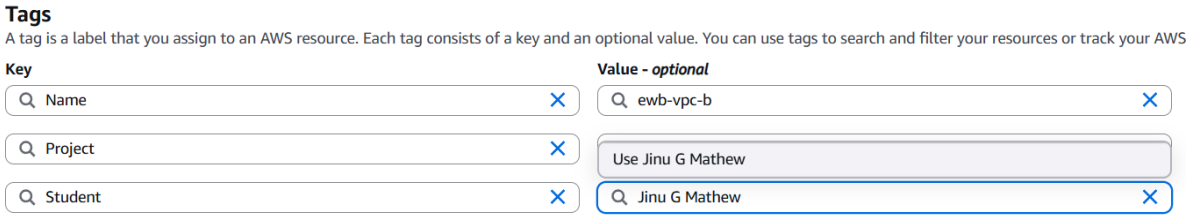


Figure 14: Tags on ewb-vpc-b: Name, Project and Student.

4.2 Subnet and route table

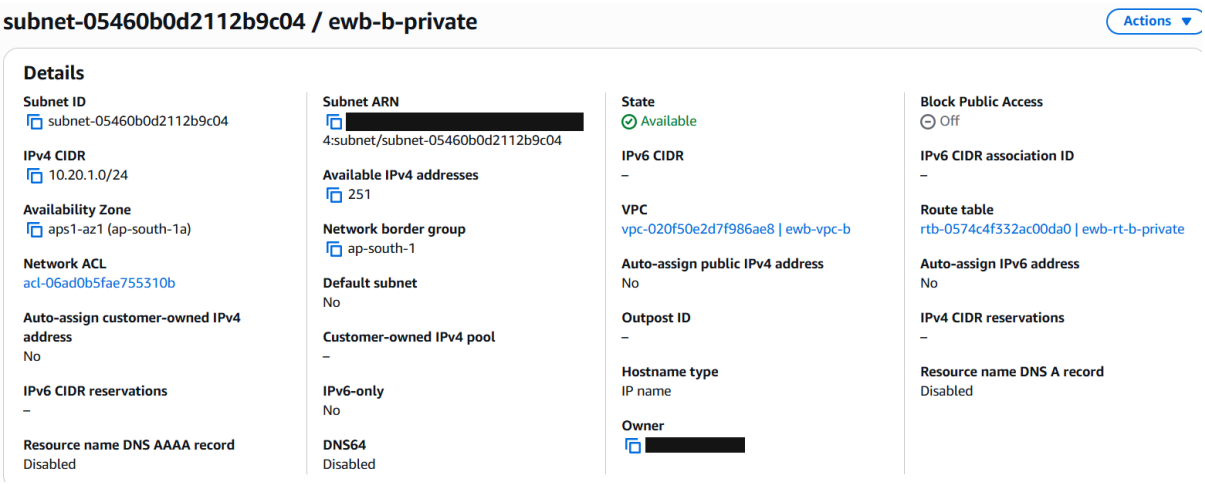


Figure 15: ewb-b-private (subnet-05460b0d2112b9c04): 10.20.1.0/24 in ap-south-1a, VPC ewb-vpc-b (vpc-020f50e2d7f986ae8), route table ewb-rt-b-private, auto-assign public IPv4 off.

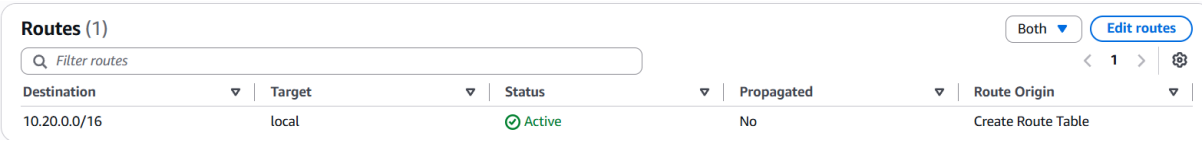


Figure 16: ewb-rt-b-private (rtb-0574c4f332ac00da0), Routes tab (before peering): only 10.20.0.0/16 local. VPC B has no internet gateway.

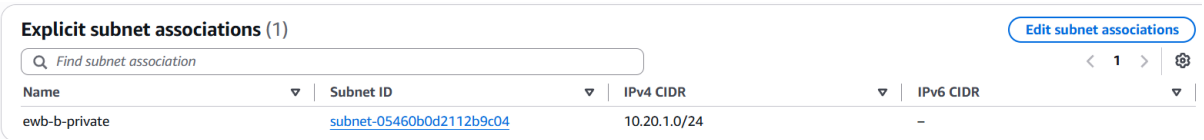


Figure 17: ewb-rt-b-private, Subnet associations tab: ewb-b-private is the explicit association.

5. Security groups

Three security groups restrict inbound traffic to the lab hosts only. No rule uses 0.0.0.0/0. Outbound rules were left at the default (allow all), and because security groups are stateful, replies to permitted traffic are allowed automatically.

Security group	VPC	Inbound rules
ewb-sg-bastion	ewb-vpc-a	SSH, TCP 22 from my public IPv4 /32
ewb-sg-app-a	ewb-vpc-a	SSH, TCP 22 from 10.10.1.10/32 (bastion); ICMP Echo Request from 10.20.1.10/32 (App-B)
ewb-sg-app-b	ewb-vpc-b	SSH, TCP 22 from 10.10.1.10/32 (bastion); ICMP Echo Request from 10.10.2.10/32 (App-A)

5.1 ewb-sg-bastion

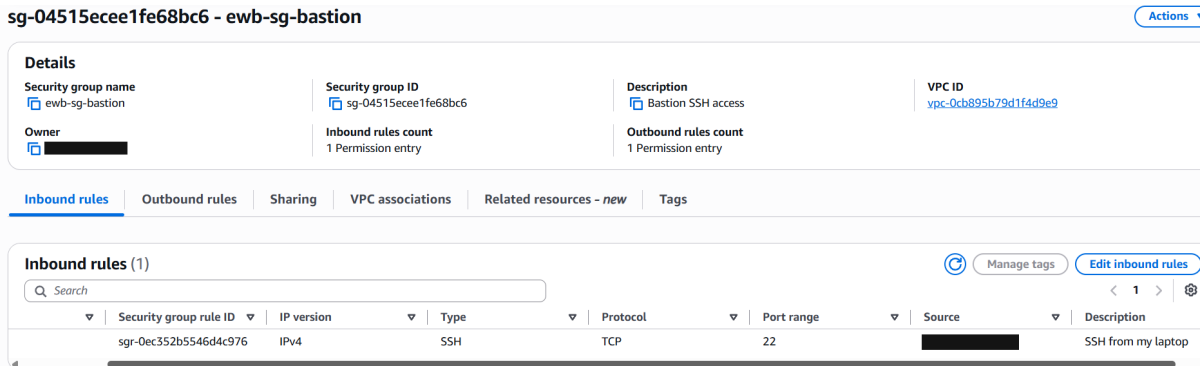


Figure 18: ewb-sg-bastion (sg-04515ecee1fe68bc6) in ewb-vpc-a: one inbound rule, SSH TCP 22 from my single public IPv4 (/32). My ISP changed my public IP during the lab, so I edited this rule before launching the instances. The IP is hidden in this public copy.

5.2 ewb-sg-app-a

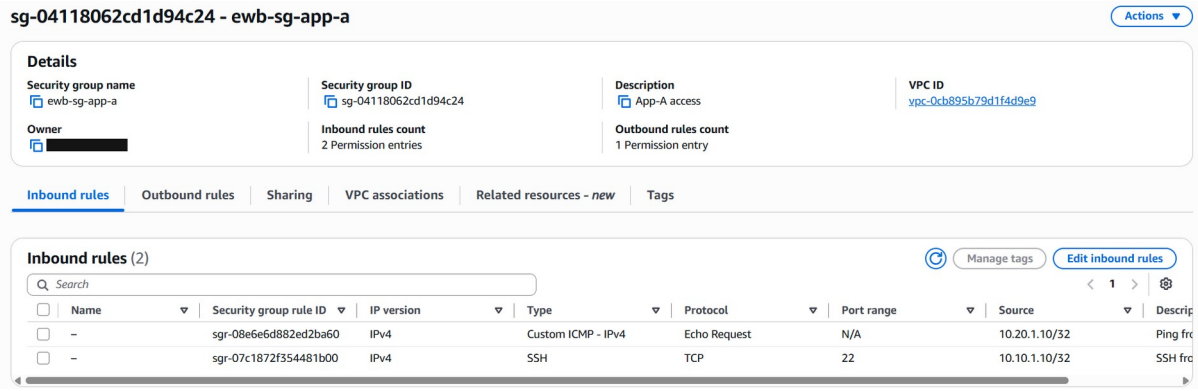


Figure 19: ewb-sg-app-a (sg-04118062cd1d94c24) in ewb-vpc-a. SSH from the bastion 10.10.1.10/32 and ICMP Echo Request from App-B 10.20.1.10/32.

5.3 ewb-sg-app-b

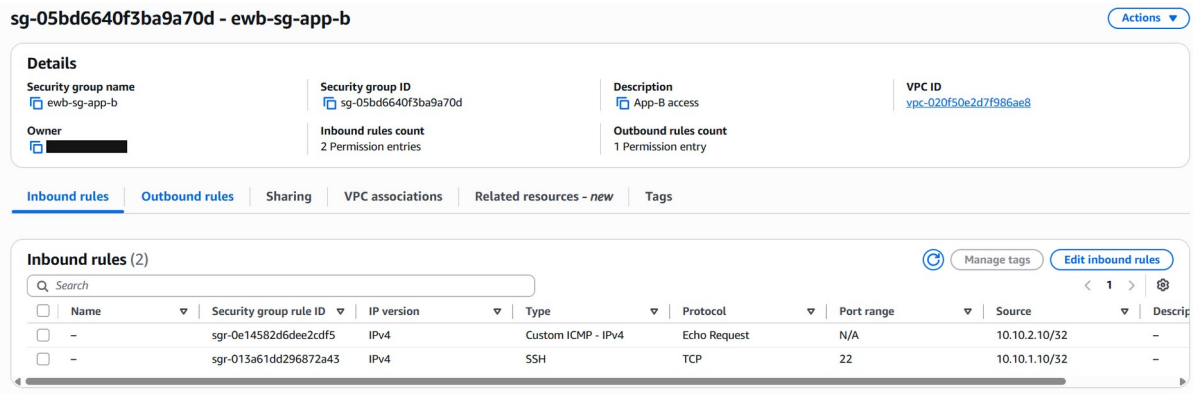


Figure 20: ewb-sg-app-b (sg-05bd6640f3ba9a70d) in ewb-vpc-b (vpc-020f50e2d7f986ae8). SSH from the bastion 10.10.1.10/32 and ICMP Echo Request from App-A 10.10.2.10/32.

App-B receives SSH from the bastion's private IP, because the bastion opens a separate SSH connection to App-B when ProxyJump is used.

6. Key pair

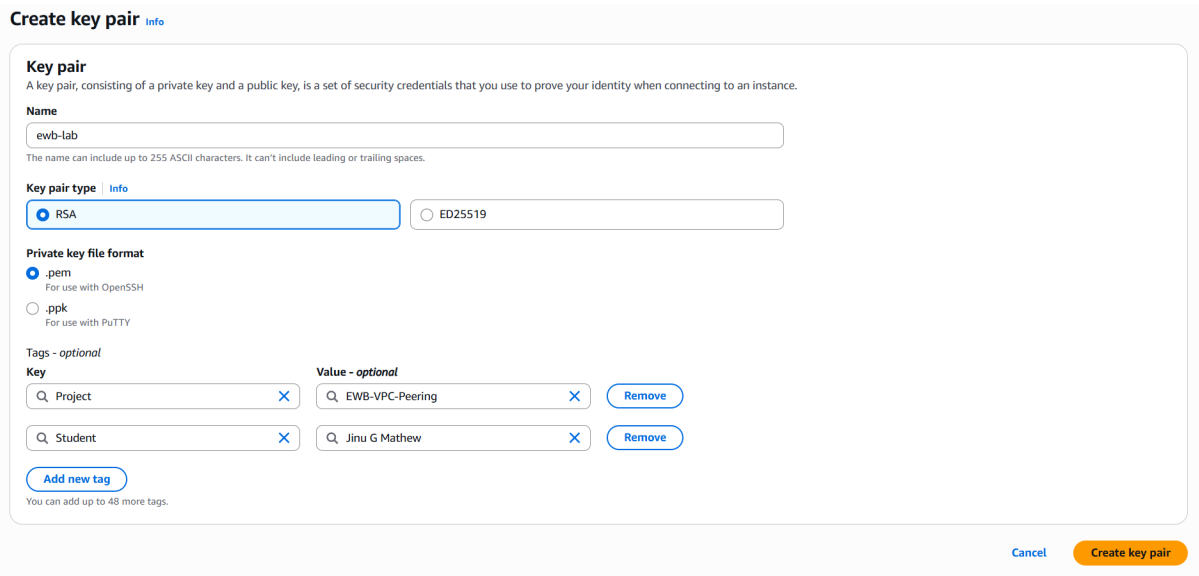


Figure 21: Creating key pair ewb-lab: type RSA, private key format .pem, with Project and Student tags.

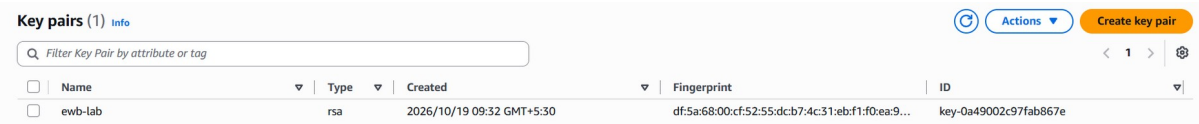


Figure 22: Key pairs list showing ewb-lab (RSA, key-0a49002c97fab867e). The private key file was stored only on my laptop and is not included in this report.

7. EC2 instances (Checkpoint 02)

Three Amazon Linux 2023 instances (t2.micro, 8 GiB root volume, delete on termination enabled) were launched with the key pair ewb-lab and fixed private IPs. Only the bastion received a public IPv4 address. The CPU credit specification was set to Standard because the account could not launch T2 instances with Unlimited enabled.

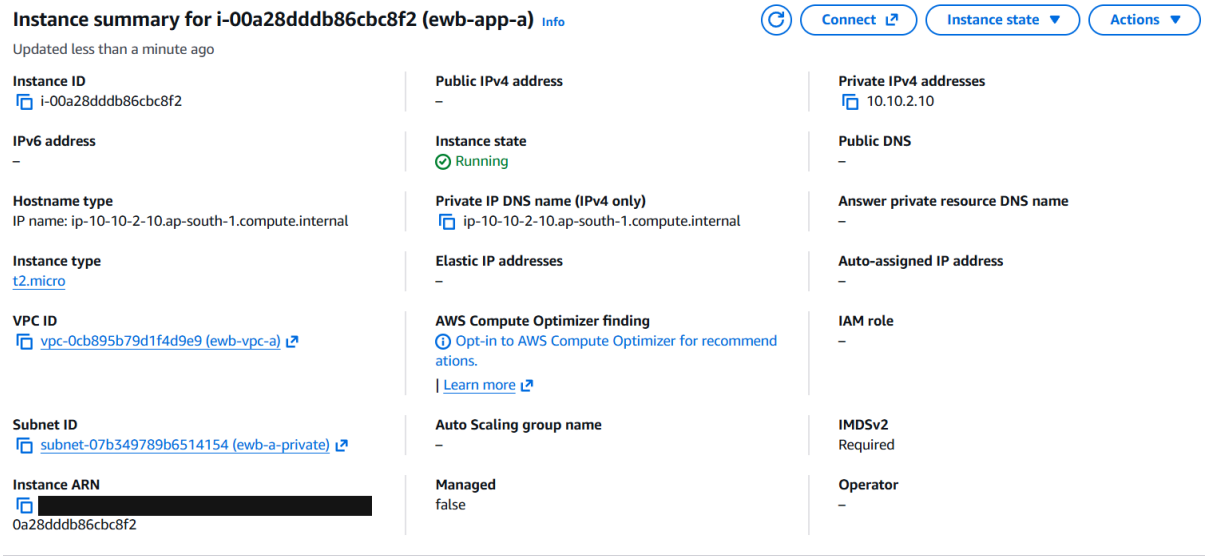


Figure 23: ewb-app-a (i-00a28dddb86cbc8f2): Running, private IP 10.10.2.10, Public IPv4 address empty, VPC ewb-vpc-a, subnet ewb-a-private.

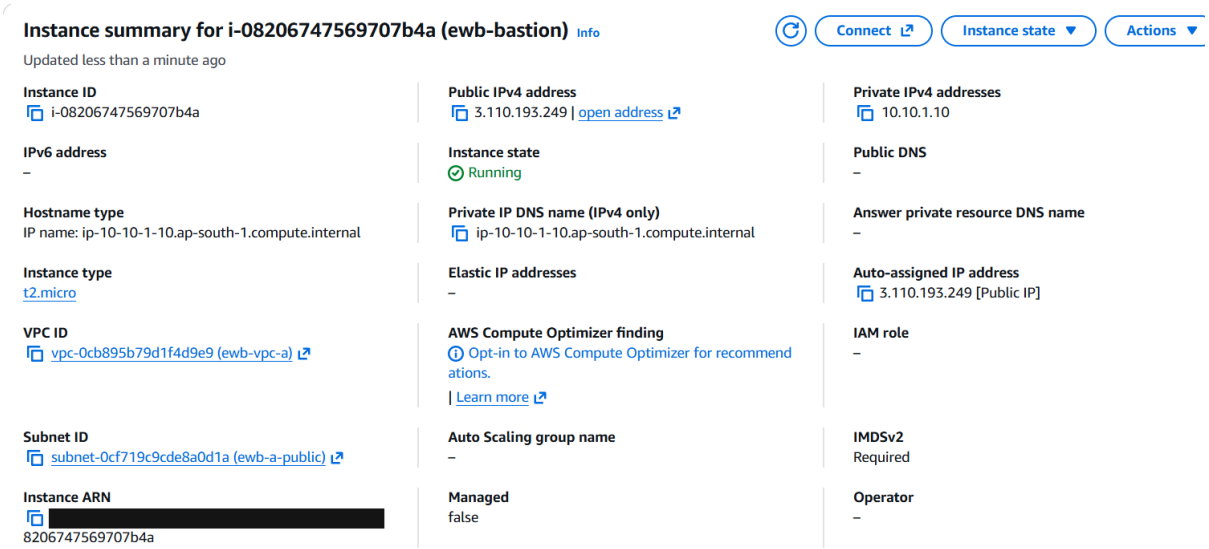


Figure 24: ewb-bastion (i-08206747569707b4a): Running, public IPv4 3.110.193.249, private IP 10.10.1.10, subnet ewb-a-public.

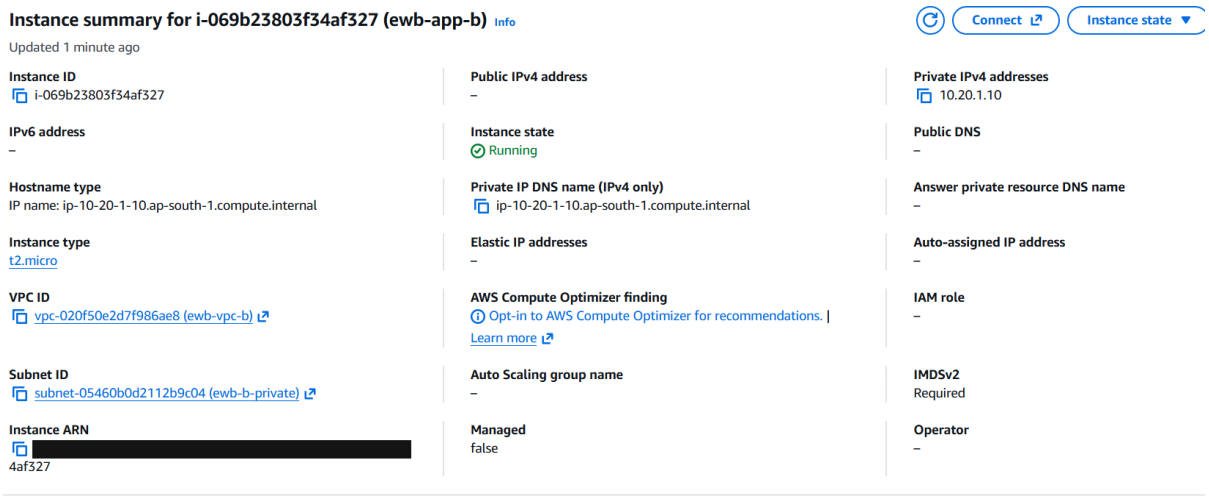


Figure 25: ewb-app-b (i-069b23803f34af327): Running, private IP 10.20.1.10, Public IPv4 address and Public DNS empty, VPC ewb-vpc-b, subnet ewb-b-private.

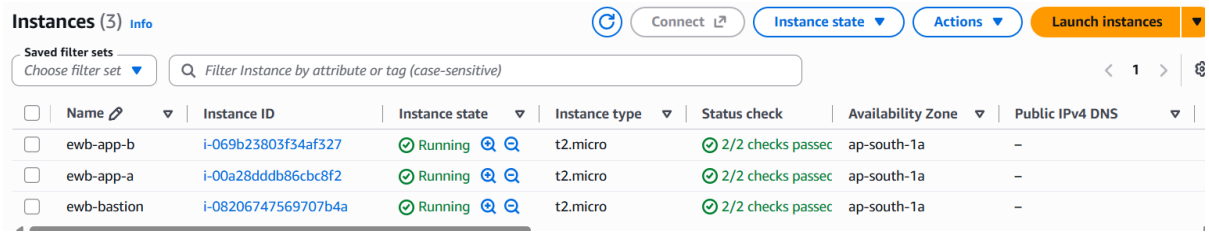


Figure 26: All three instances Running with 2/2 status checks passed, in ap-south-1a.

8. SSH access and baseline test (before peering)

The OpenSSH client built into Windows PowerShell was used. The key was copied into my .ssh folder and its permissions were restricted, then a config file defined ewb-bastion, ewb-app-a and ewb-app-b, with ProxyJump through the bastion for the two private hosts.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> ssh -v
usage: ssh [-46AaCfGgKkMNnqsTtVvXxYy] [-B bind_interface] [-b bind_address]
           [-c cipher_spec] [-D [bind_address:]port] [-E log_file]
           [-e escape_char] [-F configfile] [-I pkcs11] [-i identity_file]
           [-J destination] [-L address] [-l login_name] [-m mac_spec]
           [-O ctl_cmd] [-o option] [-P tag] [-p port] [-Q query_option]
           [-R address] [-S ctl_path] [-W host:port] [-w local_tun[:remote_tun]]
           destination [command [argument ...]]
PS C:\Windows\system32> New-Item -ItemType Directory -Force $env:USERPROFILE\.ssh


    Directory: C:\Users\Jinu G Mathew


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        19-10-2026     10:20                .ssh


PS C:\Windows\system32> Copy-Item $env:USERPROFILE\Downloads\ewb-lab.pem $env:USERPROFILE\.ssh\ewb-lab.pem
PS C:\Windows\system32> Get-ChildItem "$env:USERPROFILE\.ssh"


    Directory: C:\Users\Jinu G Mathew\.ssh


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        19-10-2026     09:33           1674 ewb-lab.pem


PS C:\Windows\system32> icacls "$env:USERPROFILE\.ssh\ewb-lab.pem" /inheritance:r
processed file: C:\Users\Jinu G Mathew\.ssh\ewb-lab.pem
Successfully processed 1 files; Failed processing 0 files
PS C:\Windows\system32> icacls "$env:USERPROFILE\.ssh\ewb-lab.pem" /grant:r "$($env:USERNAME):(R)"
processed file: C:\Users\Jinu G Mathew\.ssh\ewb-lab.pem
Successfully processed 1 files; Failed processing 0 files
PS C:\Windows\system32> notepad "$env:USERPROFILE\.ssh\config"
```

Figure 27: Key stored in the .ssh folder with restricted permissions (icacls), and the SSH config file opened for editing. The key contents are not shown.

```
PS C:\Windows\system32> Rename-Item "$env:USERPROFILE\.ssh\config.txt" config
PS C:\Windows\system32> ssh ewb-bastion
The authenticity of host '3.110.193.249 (3.110.193.249)' can't be established.
ED25519 key fingerprint is SHA256:cj9u8vKzWhJAwoAScxNbcZZKR5VcuVGvFtotj0ftJdw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.110.193.249' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-1-10 ~]$
```

Figure 28: ssh ewb-bastion: connected to the bastion (prompt ec2-user@ip-10-10-1-10).

Test 01: Baseline (App-A to App-B before peering)

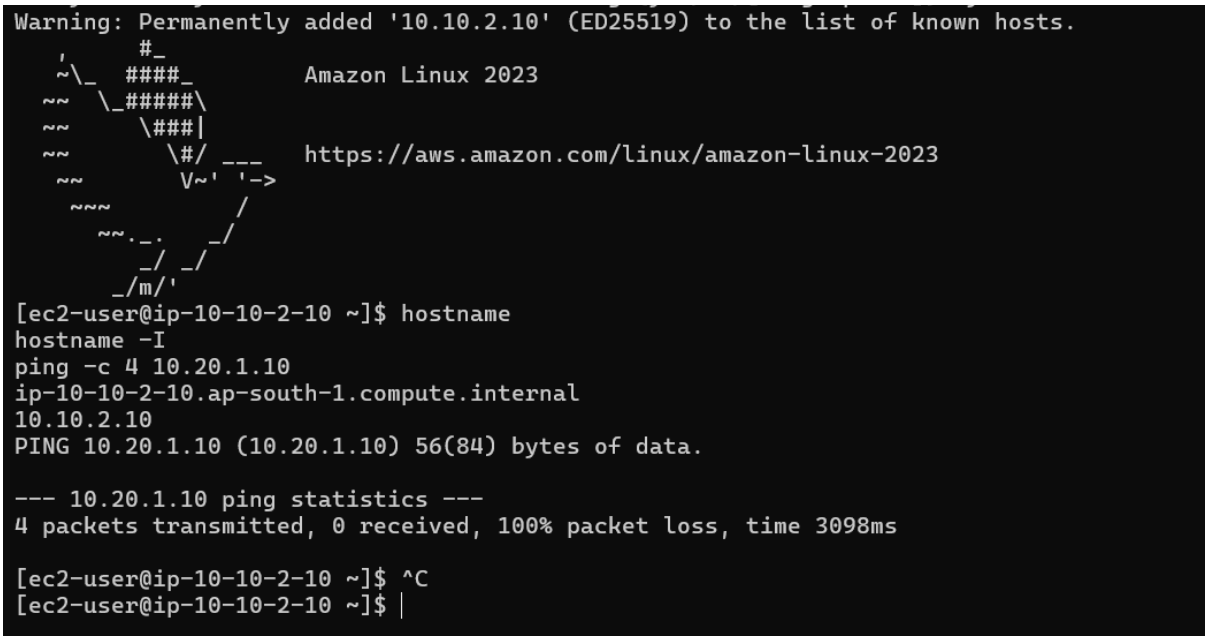


Figure 29: ewb-app-a (10.10.2.10): ping -c 4 10.20.1.10 before peering. 4 transmitted, 0 received, 100% packet loss.

9. Peering connection (Checkpoint 03)

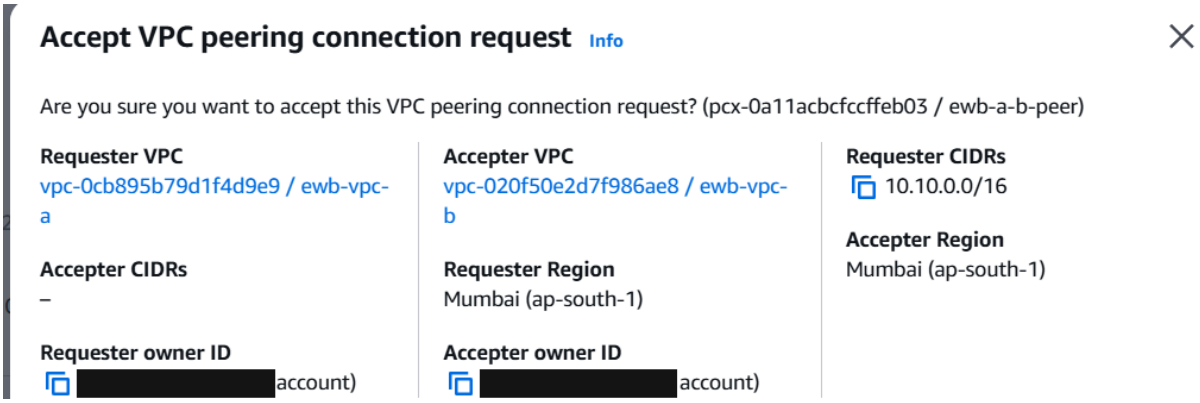


Figure 30: Accepting the request: pcx-0a11acbcfccffeb03 / ewb-a-b-peer, requester ewb-vpc-a, accepter ewb-vpc-b, same account and Region. The request must be accepted even though both VPCs are mine.

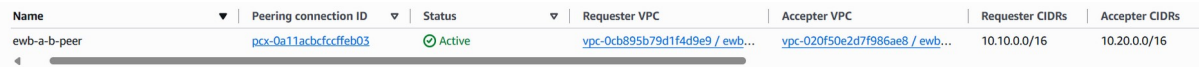


Figure 31: ewb-a-b-peer is Active. Requester VPC vpc-0cb895b79d1f4d9e9 (10.10.0.0/16), accepter VPC vpc-020f50e2d7f986ae8 (10.20.0.0/16).

Active peering alone does not let traffic flow. Routes in both VPCs are needed, as shown in the next section.

10. Peering routes (Checkpoint 04)

One route to the other VPC was added to each route table. A route destination is always the other VPC's range. Both directions are needed so that replies can return. My first attempt added 10.20.0.0/16 to a VPC B table, which AWS rejected because it is VPC B's own range; I corrected it to 10.10.0.0/16.

Route table	Added route	Purpose
ewb-rt-a-private	10.20.0.0/16 to pcx	App-A to VPC B
ewb-rt-b-private	10.10.0.0/16 to pcx	Return path to VPC A
ewb-rt-a-public	10.20.0.0/16 to pcx	Bastion SSH to App-B

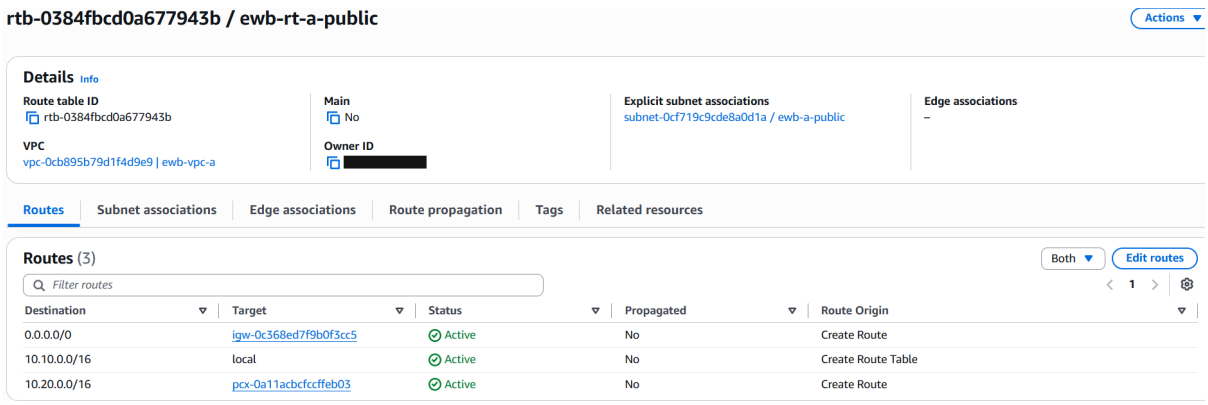


Figure 32: ewb-rt-a-public (VPC A): routes 0.0.0.0/0 to igw, 10.10.0.0/16 local and 10.20.0.0/16 to pcx-0a11acbcfccffeb03, all Active. Explicit subnet association: ewb-a-public.

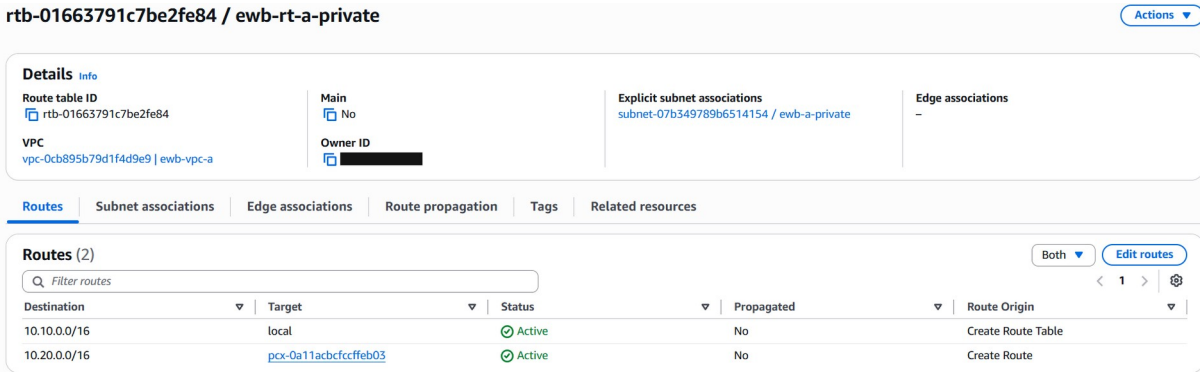


Figure 33: ewb-rt-a-private (VPC A): 10.10.0.0/16 local and 10.20.0.0/16 to pcx-0a11acbcfccffeb03, both Active. Explicit subnet association: ewb-a-private.

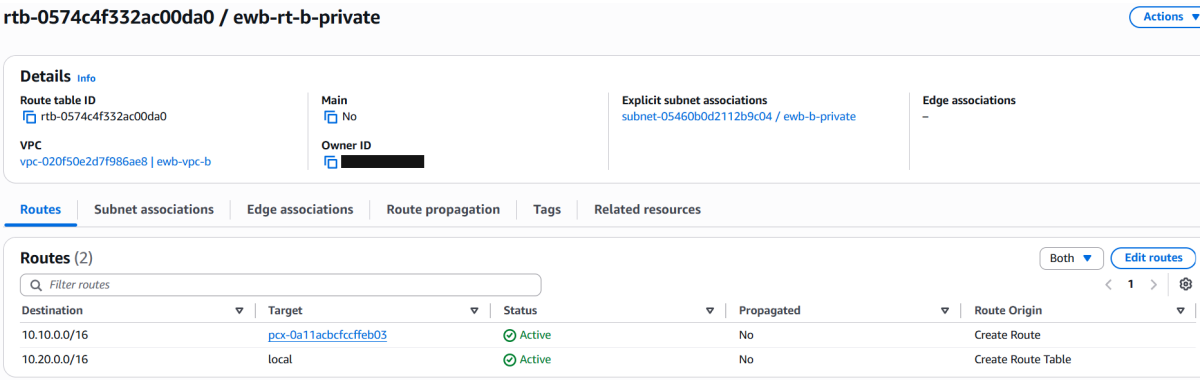


Figure 34: ewb-rt-b-private (VPC B): 10.10.0.0/16 to pcx-0a11acbcfccffeb03 and 10.20.0.0/16 local, both Active. Explicit subnet association: ewb-b-private.

11. Validation tests

Test 02: Forward path (App-A to App-B)

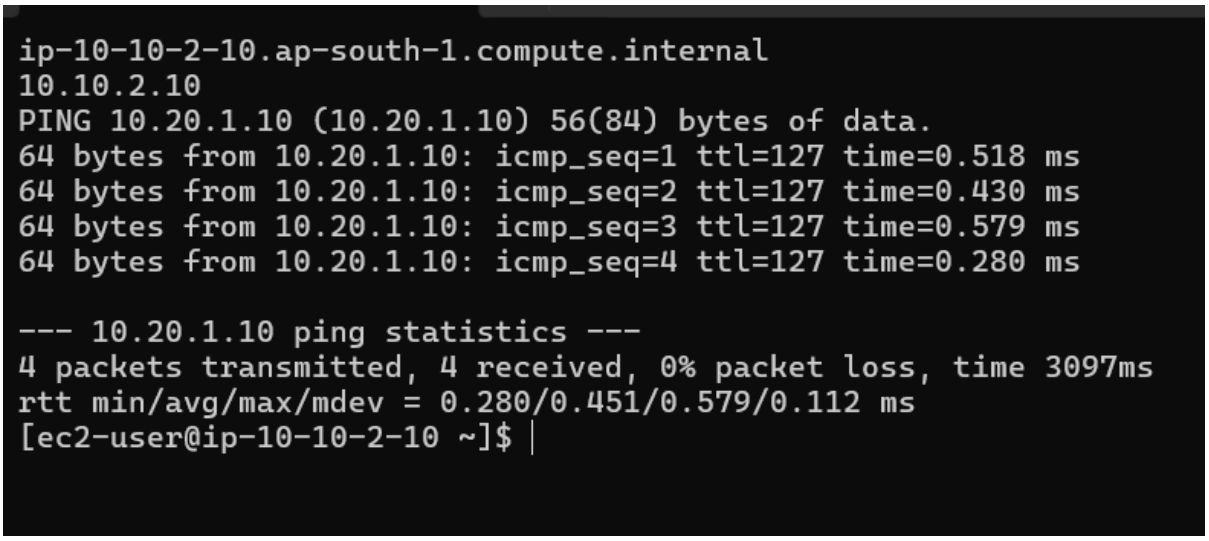


Figure 35: ewb-app-a (10.10.2.10): ping -c 4 10.20.1.10 after peering and routes. 4 transmitted, 4 received, 0% packet loss.

Test 04: Private SSH to App-B through the bastion

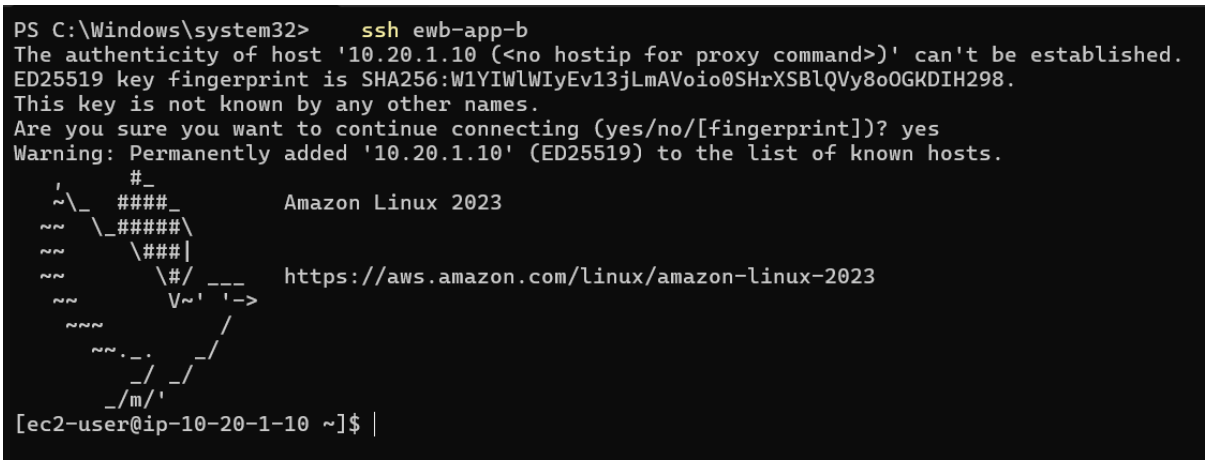


Figure 36: ssh ewb-app-b from my laptop: shell on App-B (ec2-user@ip-10-20-1-10) through the bastion using ProxyJump.

Test 03: Reverse path (App-B to App-A)

```
[ec2-user@ip-10-20-1-10 ~]$ hostname
ip-10-20-1-10.ap-south-1.compute.internal
[ec2-user@ip-10-20-1-10 ~]$ hostname -I
10.20.1.10
[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.257 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.282 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.290 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.918 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3130ms
rtt min/avg/max/mdev = 0.257/0.436/0.918/0.278 ms
[ec2-user@ip-10-20-1-10 ~]$
```

Figure 37: ewb-app-b (10.20.1.10): ping -c 4 10.10.2.10. 4 transmitted, 4 received, 0% packet loss.

Test 05: Isolation

Neither private host has a public IPv4 address: see Section 7, where the details pages of ewb-app-a and ewb-app-b show Public IPv4 address and Public DNS empty. All tests used private destination IPs.

Test	Where / command	Result
01 Baseline	App-A: ping -c 4 10.20.1.10 (before peering)	100% packet loss
02 Forward path	App-A: ping -c 4 10.20.1.10 (after routes)	4 replies, 0% loss
03 Reverse path	App-B: ping -c 4 10.10.2.10 (after routes)	4 replies, 0% loss
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B through bastion
05 Isolation	EC2 instance details: App-A, App-B	No public IPv4 on either host

12. Controlled failure and recovery

To show that the security group is what permits the ping, I removed only the ICMP Echo Request rule from ewb-sg-app-b (keeping the SSH rule), tested, then restored the same rule and tested again. The peering connection and routes were not changed.

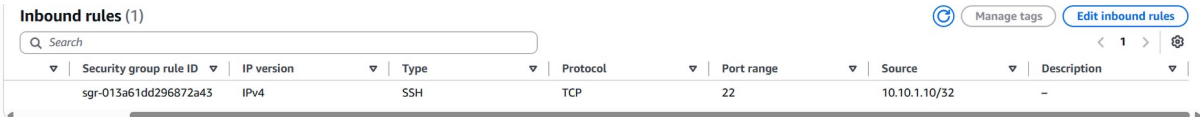


Figure 38: ewb-sg-app-b with the ICMP rule removed: 1 inbound rule remains, SSH TCP 22 from 10.10.1.10/32.

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3090ms

[ec2-user@ip-10-10-2-10 ~]$
```

Figure 39: ewb-app-a: ping -c 4 10.20.1.10 with the ICMP rule removed. 4 transmitted, 0 received, 100% packet loss.

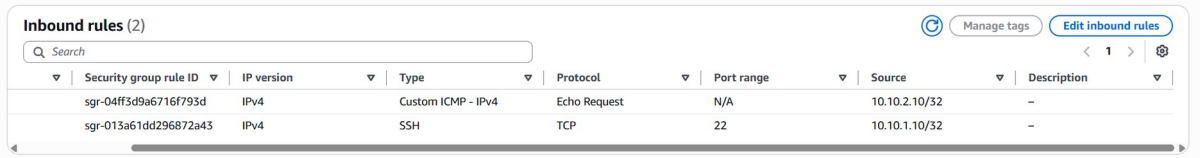


Figure 40: ewb-sg-app-b with the rule restored: 2 inbound rules, ICMP Echo Request from 10.10.2.10/32 and SSH from 10.10.1.10/32.

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.904 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.270 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.541 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.331 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3093ms
rtt min/avg/max/mdev = 0.270/0.511/0.904/0.247 ms
[ec2-user@ip-10-10-2-10 ~]$
```

Figure 41: ewb-app-a: ping -c 4 10.20.1.10 after restoring the rule. 4 transmitted, 4 received, 0% packet loss.

Explanation: the route and peering were unchanged throughout, so the only difference was the destination security group. Removing the inbound ICMP Echo Request rule made App-B drop the pings, and restoring it brought the replies back.

13. Forward path and return path

Forward path: App-A (10.10.2.10) sends a packet to 10.20.1.10. The subnet's route table ewb-rt-a-private matches 10.20.0.0/16 and sends it to the peering connection pcx-0a11acbcfccffeb03. In VPC B, the security group ewb-sg-app-b allows ICMP Echo Request from 10.10.2.10/32, so App-B accepts it.

Return path: App-B replies to 10.10.2.10. The route table ewb-rt-b-private has 10.10.0.0/16 pointing to the peering connection, so the reply goes back to VPC A. Security groups are stateful, so the reply is allowed automatically.

14. Cleanup evidence

After testing, all resources created for the project were removed in this order: the three instances (and their volumes), the peering routes and ewb-a-b-peer, the security groups, subnets, route tables, internet gateway, both VPCs and finally the key pair and local key file. Only the account's default VPC resources remain.

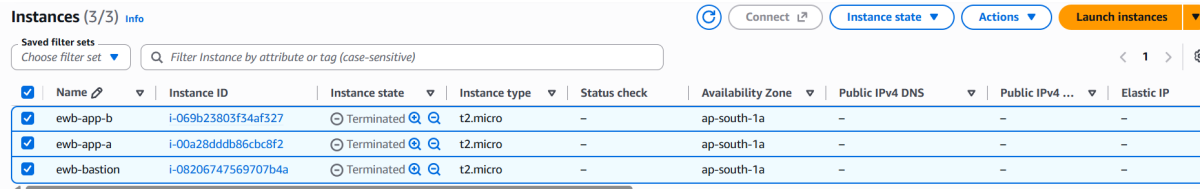


Figure 42: All three instances (ewb-app-b, ewb-app-a, ewb-bastion) show Terminated.

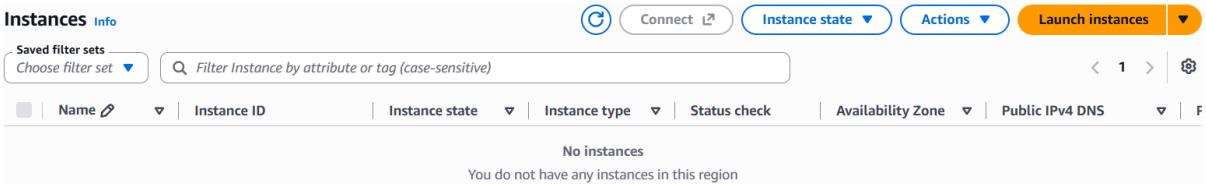


Figure 43: Instances list is empty in ap-south-1.

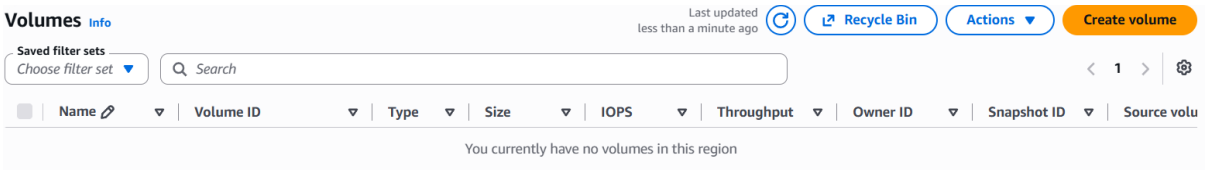


Figure 44: Volumes: no volumes remain, so delete on termination removed the root disks.

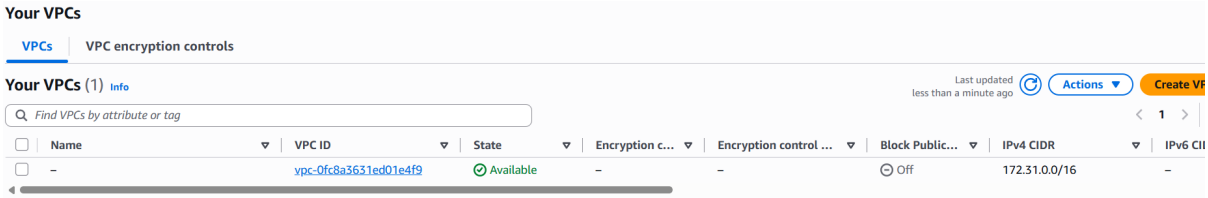


Figure 45: Your VPCs: only the default VPC (172.31.0.0/16) remains. ewb-vpc-a and ewb-vpc-b are deleted.

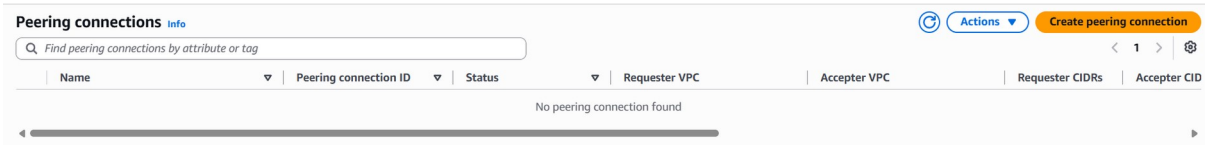


Figure 46: Peering connections: none found. ewb-a-b-peer is deleted.

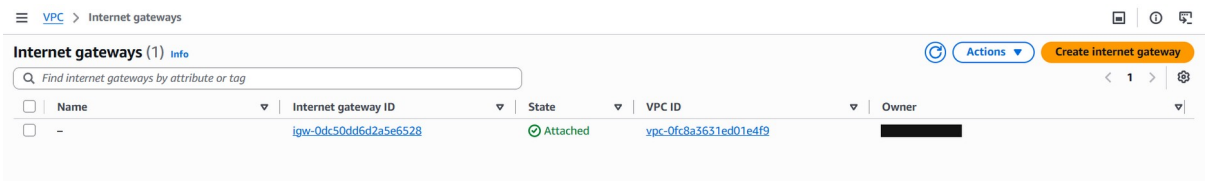


Figure 47: Internet gateways: only the default VPC's gateway remains. ewb-igw-a is deleted.

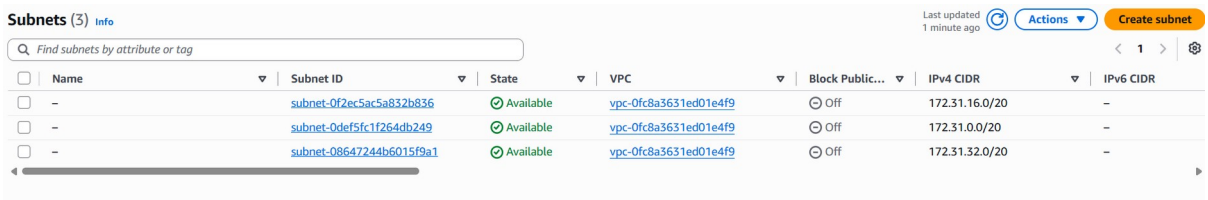


Figure 48: Subnets: only the three default VPC subnets remain.

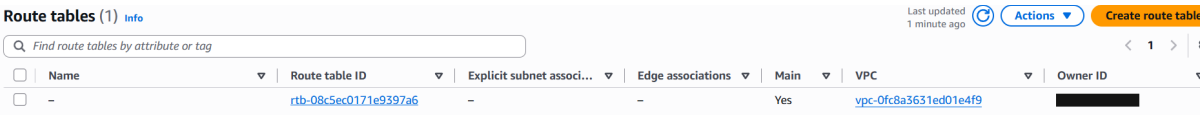


Figure 49: Route tables: only the default VPC's main route table remains.

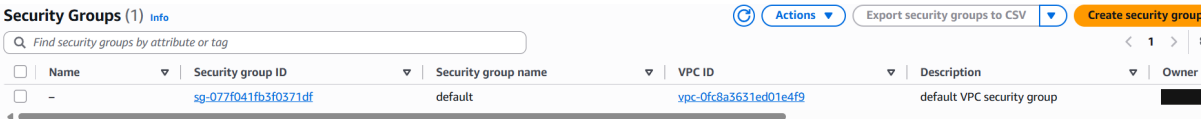


Figure 50: Security groups: only the default security group remains.

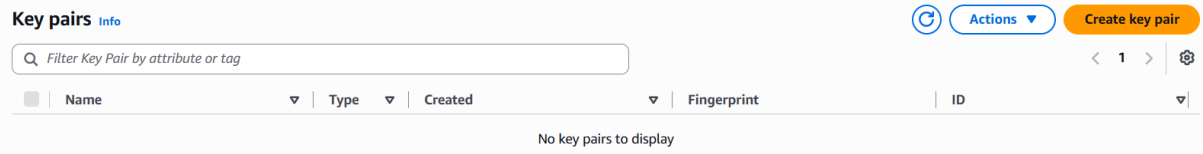


Figure 51: Key pairs: none. ewb-lab is deleted (the local .pem file was also removed).

15. What I learned

I learned that a VPC peering connection is only the link: traffic flows only when both VPCs have routes to each other's CIDR range and the destination security group allows it. The baseline test failed with 100% loss before peering, worked in both directions afterwards, and failed again when I removed the ICMP rule, which proved each layer separately.

I also learned that a route's destination is always the other VPC's range. AWS rejected my first attempt, because I added 10.20.0.0/16 to a route table inside VPC B. A private subnet needs no internet route, and the bastion needs the peering route in its public route table to reach App-B through ProxyJump.

Other practical lessons: my public IP changed during the lab, so the bastion's /32 rule had to be updated; the account could not launch T2 instances with Unlimited CPU credits, so I set Standard; and keeping consistent names and tags made cleanup quick and easy to verify. I terminated the instances immediately after testing to keep costs minimal.